Week 8: The Machine That Could Learn

Weighted Sums

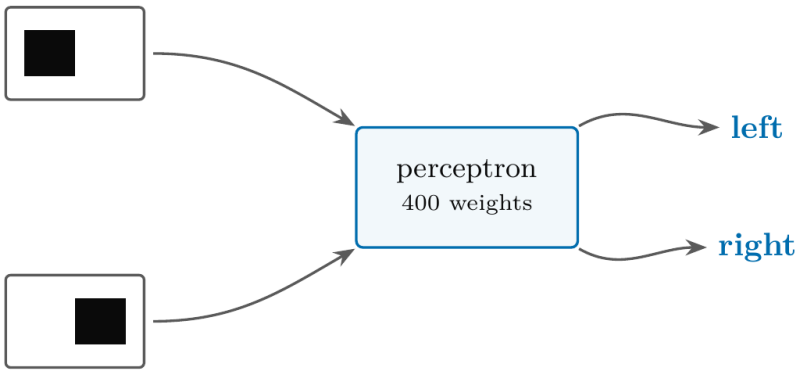
FIT1061: Introduction to Artificial Intelligence

2026-08-31

The Story: Rosenblatt's Perceptron

In the summer of **1958**, in Washington D.C., a 29-year-old researcher named **Frank Rosenblatt** stood in front of a room of reporters. Rosenblatt worked at the **Cornell Aeronautical Laboratory**, and the reporters had been gathered by the U.S. Navy's **Office of Naval Research** to see a machine called the **Perceptron**. The demonstration was simple to describe. Rosenblatt had a deck of **flashcards**, each printed with a small coloured square: some cards with the square on the **left**, some with it on the **right**. The machine's only job was to look at a card and say which side the square was on.

What made this more than a party trick is that nobody had told the machine the rule. It was not programmed with "if the square is on the left, say left." Instead it **learned** the rule from examples. And once it had seen enough cards, it could correctly label cards **it had never seen before**.



Learns the rule from ∼**50** labelled cards — then labels cards it has **never seen**.

Figure 1: The Mark I Perceptron's task: a flashcard with a square on the left or the right becomes 400 numbers; a single weighted sum turns those numbers into a decision. After about 50 example cards, the machine labels cards it has never seen.

What was actually inside

The machine that did this is, at heart, the same machine you will build this week. A card was turned into a small image: a **20 × 20 grid = 400 pixels**, each pixel either black or white, so each card became **400 numbers**. Every one of those 400 numbers was **multiplied by its own weight** (a weight that could be positive or negative) and then **all 400 products were added together**. The machine then looked at the sign of that single total. In the narrator's words:

> "If the total is negative, it will output a minus one… if it's positive, it will output a positive one."

That is the entire decision: **multiply each input by a weight, add them up, and read off the sign.** This is not new even in 1958; it is "a rudimentary neural network, the kind that **McCulloch and Pitts** had imagined in the early 1940s." What was new was the *learning*. The weights began as random numbers, so at first the machine's answers were nonsense. Every time it guessed wrong, Rosenblatt corrected it by **dialing up the weights that were too low and turning down the weights that were too high**. After only **about 50 trials**, the weights had settled into a pattern that worked: "a self-induced change in the wiring diagram." (This week you build the *deciding* part and set the weights by hand; the rule that lets the machine adjust its own weights comes later.)

The promise, and the exaggeration

Rosenblatt himself made a careful, measured claim: for the first time, he said, "a non-biological object will achieve an **organization of its external environment in a meaningful way**." The press was not so restrained. The *New York Times* reported that

> "The Navy revealed the embryo of an electronic computer today that it expects will be able to **walk, talk, see, write, reproduce itself, and be conscious of its existence**."

The *New Yorker* went further, calling the perceptron "capable of **original thought**" and "the **first serious rival to the human brain** ever devised." Hold that gap in mind: a machine that had learned to tell a square-on-the-left from a square-on-the-right, described as a conscious rival to the human brain. The distance between **what it did** and **what was claimed** is the seed of everything that happened next.

The crash: "sterile"

In **1969**, two prominent researchers at MIT (**Marvin Minsky** and **Seymour Papert**) published a book titled *Perceptrons*. In it they proved, mathematically, that a single perceptron has a hard limit: there are simple patterns it can **never** learn, no matter how you set its weights. (We will meet one of those patterns at the end of these lecture notes.) They further speculated that stacking perceptrons into layers to escape the limit would be, in their word, **"sterile"**, though they conceded "perhaps some interesting convergence theorem [could] be discovered." The effect on the field was immediate and severe. As one account puts it, the book "effectively **slams the door shut on the entire vein of research**." Minsky himself later observed:

> "There had been several thousand papers published on perceptrons up to 1969… Our book put a stop to those."

The money followed the mood. In **1973** the British government's **Lighthill report** judged that AI research had not delivered what it promised, and UK funding for the field was largely stopped. American support fell away over the same years, less by one decision than by a steady withdrawal of defence money. The field went quiet for over a decade: a stretch now called the **AI winter**.

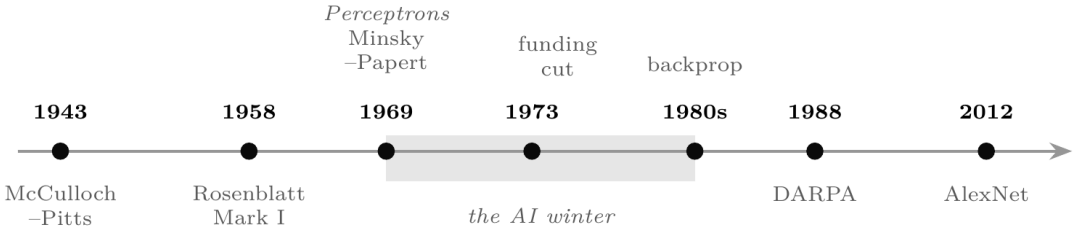


Figure 2: From the first artificial neuron to the modern revival. The neural-network idea was proved limited in 1969 and lost its funding by 1973 (the shaded winter), then returned in stages: a way to train stacked layers in the 1980s, renewed attention in 1988, and a practical explosion in 2012.

The staged revival

The comeback was **not a single moment** but a sequence of them:

1. **Backpropagation, 1980s.** By the late 1970s and early 80s, several groups (among them **David Rumelhart** and **Geoffrey Hinton**) had worked out a general method for training networks with *stacked* layers, directly rebutting Minsky and Papert's "sterile" verdict. The precise thing they solved was the gap Minsky and Papert had identified: how to tune the weights of the *inner* layers, not just the last one.
2. **DARPA, 1988.** A top official at the U.S. defence research agency **DARPA** proclaimed that neural networks were "**more important than the atom bomb**." Suddenly the field was fashionable again.
3. **AlexNet, 2012.** The mathematics had been ready since the 1980s, but the practical breakthrough waited on **data and computing power**: specifically the fast graphics chips built for computer games. A large network trained on those chips won a major image-recognition contest and, in one observer's phrase, "obliterated the rest of the entire field."

Minsky, decades on, was gracious about it: "**I now believe**," he admitted, "**that the book was overkill.**" The lesson worth carrying forward is that **every modern network** (the image classifiers, the language models behind today's chatbots) is nothing more exotic than **perceptrons stacked in layers**. Understand the single perceptron and you understand the atom from which all of it is built. The rest of these lecture notes are about that single perceptron.

The Weighted Sum

Strip away the history and a perceptron is one small piece of arithmetic. It takes an input described by a few numbers (a **feature vector** $\mathbf{x} = (x_1, x_2, \dots)$) and it holds a **weight** w_i for each feature. The core operation is to multiply each feature by its weight and add the results. That sum has a name: the **dot product**. It is new this week, and nothing earlier in the unit depends on it.

$$\mathbf{w} \cdot \mathbf{x} = w_1 x_1 + w_2 x_2 + \dots$$

Last week's filter was already a line

The lecture opened with a warm-up that is worth keeping in view. Take a Naive Bayes spam filter like last week's, on a small inbox of eight training messages over five tracked words. Its rule compares two scores. Divide one by the other and the rule becomes "predict spam when the ratio is above 1", and because each score is a product, the ratio is a product of one factor per word: how much more often that word appears in spam than in ham. In the warm-up's inbox every spam-leaning word had ratio 2 and every ham-leaning word had ratio 1/2.

Take $\log_2$ of both sides. A log turns a product into a sum, and $\log_2$ counts doublings, so each spam-leaning word contributes $+1$, each ham-leaning word -1, and "above 1" becomes "above 0". With x_1 the number of spam-leaning words in a message and x_2 the number of ham-leaning words,

$$\log_2 \frac{\text{score(spam)}}{\text{score(ham)}} = (+1)\, x_1 + (-1)\, x_2,$$

which is a weighted sum with weights $\mathbf{w} = (1, -1)$ and bias 0. Its verdict flips on the line $x_2 = x_1$. Naive Bayes set those weights by counting, and never checked the line against the picture of the data. Many other lines fit the same eight messages equally well, and they disagree about the next message. That is the question this week opens: what if you could set the weights yourself?

In general a word's weight is the log of its ratio, in any base, and there is one weight per word. The warm-up collapsed to two axes only because every word in each group shared the same ratio.

The dot product measures agreement

It helps to read the dot product not as a formula but as a measure of **agreement** between two arrows: the weight arrow $\mathbf{w}$ and the input arrow $\mathbf{x}$.

- When the two arrows point in **similar directions** (a small angle between them), the dot product is **positive**.
- When they are **perpendicular**, the dot product is **zero**.
- When they point in **opposing directions**, the dot product is **negative**.

You can check all three by hand. Hold $\mathbf{w} = (4, 1)$ fixed and move $\mathbf{x}$:

$\mathbf{x}$	$4x_1 + 1x_2$		
$(3, 3)$	$12 + 3$	$= +15$	the arrows agree
$(-1, 4)$	$-4 + 4$	$= 0$	they are perpendicular
$(-3, -2)$	$-12 - 2$	$= -14$	they oppose

The middle row is what *perpendicular* means for a dot product, and it is the fact the decision boundary rests on a few pages from now.

So the weight vector $\mathbf{w}$ acts like a direction the neuron is "looking for." An input that lines up with that direction scores positive; an input pointing the other way scores negative. Setting the weights *is* choosing what the neuron looks for.

The decision rule

The dot product gives a number; a classifier needs a **yes-or-no answer**. We get one by adding a single extra number, the **bias** b, and then reading off the sign of the total.

Definition: the perceptron decision rule

$$\hat{y} = \begin{cases} +1 & \text{if } \mathbf{w} \cdot \mathbf{x} + b \geq 0, \\ -1 & \text{otherwise.} \end{cases}$$

Here $\hat{y}$ is the **predicted** class. The hat marks it as the model's output, as distinct from y, the **label** that came with the data point.

The bias b is the **bar the weighted sum must clear**. A large negative bias makes the neuron reluctant to say $+1$; a positive bias makes it eager. Two small details matter and are easy to miss:

- The output is ± 1, not $0/1$: the two classes are $+1$ and -1.

- The comparison is $\geq$, **not** $>$. A point that lands **exactly on the boundary**, where $\mathbf{w} \cdot \mathbf{x} + b = 0$, is classified $+1$. The tie goes to the positive class.

As a running example, take $\mathbf{w} = [\,2,\, -1\,]$ and $b = -1$. The rule becomes: compute the **score**

$$\text{score} = \mathbf{w} \cdot \mathbf{x} + b = 2x_1 - x_2 - 1,$$

and predict $+1$ if the score is ≥ 0, otherwise -1.

The boundary is a straight line

The prediction flips from -1 to $+1$ exactly where the score is zero. That set of points (everywhere satisfying $\mathbf{w} \cdot \mathbf{x} + b = 0$) is the **decision boundary**, and in two dimensions it is a **straight line**. We can see this by rearranging the equation to the familiar "$x_2 = \text{slope} \times x_1 +$ intercept" form:

$$\mathbf{w} \cdot \mathbf{x} + b = 0 \quad \Longrightarrow \quad w_1 x_1 + w_2 x_2 + b = 0 \quad \Longrightarrow \quad x_2 = -\frac{w_1}{w_2}\, x_1 - \frac{b}{w_2}.$$

> **!** Watch the minus sign
>
> The slope of the boundary is $-w_1/w_2$, with a **minus sign** out front. It is tempting to write w_1/w_2; that is the single most common slip in this whole topic. The minus comes from moving $w_1 x_1$ across to the other side of the equals sign.

For our example, $\mathbf{w} = [\,2,\, -1\,]$ and $b = -1$ give slope $-w_1/w_2 = -2/(-1) = 2$ and intercept $-b/w_2 = -(-1)/(-1) = -1$, so the boundary is the line $x_2 = 2x_1 - 1$. Everything below and to the right of that line scores $+1$; everything above and to the left scores -1.

How confident? The margin

The sign of the score tells us *which* side of the line a point is on; the **size** of the score tells us *how far* from the line it sits, and a point far from the boundary is one the classifier is more confident about. To turn the raw score into an honest distance we divide by the length of the weight vector, written $\|\mathbf{w}\|$.

> **i** Definition: the margin
>
> $$\text{margin} = \frac{\mathbf{w} \cdot \mathbf{x} + b}{\|\mathbf{w}\|}, \qquad \text{where } \|\mathbf{w}\| = \sqrt{w_1^2 + w_2^2}.$$

The margin is the **signed distance** from the point to the boundary line. Its sign is the prediction (+ for the +1 side, − for the −1 side); its magnitude is the **confidence**. A point sitting right on top of the line has margin near zero (the classifier could go either way) while a point far out in +1 territory has a large positive margin. Dividing by $\|\mathbf{w}\|$ matters because otherwise you could inflate the "confidence" just by scaling all the weights up, without moving the boundary at all.

Worked Example

Nothing makes this concrete like running it by hand. Keep $\mathbf{w} = [\,2, -1\,]$ and $b = -1$, so the score is $2x_1 - x_2 - 1$. Here are six points, each with its **label**:

point	(x_1, x_2)	label
A	$(2, 1)$	$+1$
B	$(3, 2)$	$+1$
C	$(3, 4)$	$+1$
D	$(1, 3)$	-1
E	$(0, 2)$	-1
F	$(0, 1)$	-1

Work through each one: multiply, subtract the x_2, subtract 1, then read the sign:

- **A** $(2, 1)$: $2(2) - 1 - 1 = 4 - 2 = +2 \;\Rightarrow\; \hat{y} = +1$.
- **B** $(3, 2)$: $2(3) - 2 - 1 = 6 - 3 = +3 \;\Rightarrow\; \hat{y} = +1$.
- **C** $(3, 4)$: $2(3) - 4 - 1 = 6 - 5 = +1 \;\Rightarrow\; \hat{y} = +1$.
- **D** $(1, 3)$: $2(1) - 3 - 1 = 2 - 4 = -2 \;\Rightarrow\; \hat{y} = -1$.
- **E** $(0, 2)$: $2(0) - 2 - 1 = 0 - 3 = -3 \;\Rightarrow\; \hat{y} = -1$.
- **F** $(0, 1)$: $2(0) - 1 - 1 = 0 - 2 = -2 \;\Rightarrow\; \hat{y} = -1$.

Collecting the results:

point	(x_1, x_2)	score	predicted	label
A	$(2, 1)$	$+2$	$+1$	$+1$
B	$(3, 2)$	$+3$	$+1$	$+1$
C	$(3, 4)$	$+1$	$+1$	$+1$
D	$(1, 3)$	-2	-1	-1
E	$(0, 2)$	-3	-1	-1
F	$(0, 1)$	-2	-1	-1

Every prediction matches the truth: **6 out of 6 correct**. The three +1 points all sit below the boundary line $x_2 = 2x_1 - 1$; the three −1 points all sit above it, cleanly separated by one line.

The scores also tell us about **confidence**. The weight vector has length $\|\mathbf{w}\| = \sqrt{2^2 + (-1)^2} = \sqrt{5} \approx 2.24$. Point A has score +2, so its margin is $2/\sqrt{5} \approx 0.89$: comfortably inside +1 territory. Point C has score +1, so its margin is only $1/\sqrt{5} \approx 0.45$: correct, but sitting closest to the line and therefore the least confident of the three.

Grading a worse line

Six out of six is a flattering result, and it hides something. To see what, keep the same six points but swap in a worse set of weights: $\mathbf{w} = [\,1,\, 0\,]$ and $b = -0.5$, so the score is just $x_1 - 0.5$. The second feature is ignored entirely, and the boundary becomes the vertical line $x_1 = 0.5$.

point	score $x_1 - 0.5$	predicted	label	verdict
A	+1.5	+1	+1	correct
B	+2.5	+1	+1	correct
C	+2.5	+1	+1	correct
D	+0.5	+1	−1	**wrong**
E	−0.5	−1	−1	correct
F	−0.5	−1	−1	correct

Five out of six, and the one failure is informative. Call the +1 class the one we **approve**. Then this line has three true approvals, one wrong approval (D), and nothing missed, which makes its **precision** $3/4 = 75\%$ and its **recall** $3/3 = 100\%$.

Those two numbers describe a shape, not just a score. A classifier with perfect recall and imperfect precision never turns away a case it should have accepted, and pays for that by accepting some it should have refused. It errs on the side of saying yes. Hold onto that shape, because a real classifier is about to show it.

Two Classifiers, One Dataset

The six points were invented to be easy. A more honest test is to put two different classifiers on the same real-sized problem and see whether they agree.

The problem is **loan approval**. There are 500 applications, each described by a few numbers; 350 of them are used to fit the classifiers and the remaining **150 are held back** to test on. Of those 150, **85 went on to repay** and **65 defaulted**. Approving an applicant who repays

is the outcome we want, so "approve" is the +1 class and repayment is the truth we score against.

The two contestants are last week's **Naive Bayes** classifier and this week's **weighted sum**. Naive Bayes needs a small adjustment before it can compete, because it expects to count categories rather than read continuous numbers: each feature is first sorted into **low, medium or high**, with Laplace smoothing to handle bins that never occur. That done, both classifiers see exactly the same 150 test applications.

	accuracy	precision	recall	false approvals	false rejections
Naive Bayes	81%	77%	96%	25	3
Weighted sum	85%	88%	85%	10	13

Read the accuracy column alone and the two look close, four points apart, with nothing much to choose between them. Read the last two columns and they are barely the same kind of object.

Naive Bayes is **permissive**. It approves almost every applicant who would have repaid, missing only 3 of the 85, which is where its 96% recall comes from. The price is 25 people approved who then defaulted. That is the shape of the vertical line we graded a moment ago, scaled up to real data.

The weighted sum is **conservative**. It approves only 10 applicants who go on to default, less than half of Naive Bayes's total, and it buys that by refusing 13 people who would have paid the money back.

Neither profile is the correct one. A bank worried about losses prefers the second; an applicant who would have repaid prefers the first, and so does a regulator concerned with access to credit. **Accuracy cannot settle the argument, because the two classifiers have almost the same accuracy and completely different consequences.** Choosing between them is a decision about which mistake you would rather make, and that is not a question arithmetic answers.

The Pattern It Couldn't Learn: XOR

The worked example separated cleanly because the two classes fell on opposite sides of a single straight line. Data like that is called **linearly separable**. The trouble Minsky and Papert exposed in 1969 is that **not all data is**. Here is the smallest possible example, with just four points:

(x_1, x_2)	label
$(0, 0)$	-1
$(0, 1)$	$+1$
$(1, 0)$	$+1$
$(1, 1)$	-1

The label is $+1$ exactly when the two inputs **differ**, and -1 when they **match**. This is the **XOR** pattern (short for "exclusive or"). Plot the four points as the corners of a square and a problem jumps out: the two $+1$ points sit at **opposite corners** (top-left and bottom-right) and so do the two -1 points. The two classes are diagonally interleaved.

Now try to draw a single straight line that puts both $+1$ corners on one side and both -1 corners on the other. **You cannot.** Whichever way you tilt or slide the line, it can cleanly separate at most **three of the four points**; the fourth always ends up on the wrong side. This is not a failure of effort or a matter of finding cleverer weights; it is a hard mathematical limit. No choice of $\mathbf{w}$ and b makes a single perceptron compute XOR, because a single perceptron can only ever draw **one straight line**, and one straight line is not enough here.

This is exactly the wall the field hit in 1969, and it is worth being honest about how it was eventually climbed. The fix has two parts: **stack the perceptrons into layers** so the boundary can bend rather than stay straight, and find a way to **train** those inner layers. That second part (teaching a machine to tune its own weights from its own mistakes, rather than us setting them by hand) is precisely where the unit heads next.

Accurate — For Whom?

There is one more thing the accuracy column hides, and it is the reason this topic returns.

The 150 test applicants are not interchangeable. The data records which of two **demographic groups** each applicant belongs to, and the two groups do not arrive on equal terms:

	base repay rate
Group A	about 79%
Group B	about 38%

Read that carefully, because it is easy to misread. These are rates **in the data itself**, before any classifier has been built or any prediction made. They describe the applicants, not the model. Whatever produced that gap, whether it is income, employment history, or the long shadow of earlier lending decisions, happened before the arithmetic started.

Now put a classifier on top. When the underlying rates differ this much, a model that is **accurate overall** can still distribute its mistakes very unevenly between the two groups, because "overall" is an average and averages hide their parts. A single headline number can look reassuring while one group absorbs most of the wrong rejections and the other most of the wrong approvals.

Two questions follow, and neither has an arithmetic answer:

- **Who should bear each kind of mistake?** A wrong rejection and a wrong approval fall on different people, and rarely on the same person who chose the classifier.
- **Who decides?** Someone picked the weights, the threshold, and the measure of success. That choice is not a technical detail that happens to have social consequences; it *is* the social decision, written in arithmetic.

We will not settle this here. Measuring a disparity properly needs its own machinery, which is where this unit goes next: how to compute error rates separately for each group, what the competing definitions of fairness actually require, and why some of them cannot all hold at once.

Summary

Key takeaways:

- **A perceptron is a weighted sum plus a threshold.** Multiply each feature by its weight, add them (the **dot product** $\mathbf{w} \cdot \mathbf{x}$), add a bias b, and predict $+1$ if the total is ≥ 0, otherwise -1.
- **The dot product measures agreement.** It is positive when the input arrow lines up with the weight arrow, zero when they are perpendicular, negative when they oppose. The weights choose what the neuron looks for.
- **The decision boundary is a straight line**, $\mathbf{w} \cdot \mathbf{x} + b = 0$, which rearranges to $x_2 = -\frac{w_1}{w_2} x_1 - \frac{b}{w_2}$. The slope carries a **minus sign**: the classic trap.
- **The margin is signed distance and it measures confidence.** Dividing the score by $\|\mathbf{w}\|$ turns it into an honest distance from the boundary: the sign is the prediction, the size is how sure the classifier is.
- **A single perceptron can only draw one straight line.** It works perfectly when the data is **linearly separable** (our six-point example, 6/6 correct) and fails on patterns that are not, like **XOR**, where the best any line manages is 3 out of 4.
- **The fix is layers plus a way to train them**, which is where learning proper begins next week.
- **Two classifiers with almost the same accuracy can make opposite mistakes.** On the same 150 loan applications, Naive Bayes was permissive (96% recall, 25 wrong approvals) and the weighted sum was conservative (88% precision, 13 wrong rejections). Accuracy alone cannot choose between them.

- **Accurate overall is not the same as accurate for everyone.** When two groups arrive with base repay rates of about 79% and about 38%, a single headline accuracy can hide a very uneven distribution of errors.

Key vocabulary:

Term	Meaning
Weighted sum	Each feature multiplied by its weight, all added together: the neuron's core operation
Dot product $\mathbf{w} \cdot \mathbf{x}$	$w_1x_1 + w_2x_2 + ...$; a measure of agreement between the weight and input vectors
Parameters	The numbers a model holds that fix its behaviour; for the perceptron, the weights $\mathbf{w}$ and the bias b. Set by hand this week, learned from data later
Decision boundary	The line $\mathbf{w} \cdot \mathbf{x} + b = 0$ where the prediction flips between $+1$ and -1
Linearly separable	Data whose two classes can be cleanly split by a single straight line
Margin	$(\mathbf{w} \cdot \mathbf{x} + b)/\|\mathbf{w}\|$: the signed distance from a point to the boundary; its size is the classifier's confidence
Precision / recall	Of what we approved, how much was right (precision); of the real positives, how much we caught (recall)
Base rate	How often the positive outcome occurs in the data itself, before any classifier is built

What's next: two things, in this order. First, the question the last section opened — **who a classifier's decisions fall hardest on**, and how to measure that properly rather than arguing about it. After that, the job you did by hand passes to the machine. Choosing the parameters is a **search**, in the sense of the early weeks: the candidates are settings of (w_1, w_2, b), and the score is how many points the line gets wrong. This week you are the search algorithm. Later, a rule lets the machine run that search from its own mistakes, which is the first real step into learning.

In the Lab

This week: build a single perceptron, set its weights by hand, and see both what one straight line can do and where it fails.

You will turn every idea above into working code, starting from a small set of labelled points and a weight vector you choose yourself.

1. **Build the forward pass.** Write the weighted sum ($\mathbf{w} \cdot \mathbf{x} + b$) and the threshold that turns it into a ± 1 prediction, then run it across a whole set of points at once.
2. **Draw the boundary.** From your chosen weights, work out the line $\mathbf{w} \cdot \mathbf{x} + b = 0$ (remembering the minus sign in the slope) and see which points fall on each side.
3. **Measure confidence with the margin.** Compute the signed distance from a point to the boundary and check that points far from the line come out more confident than points hugging it.
4. **Grade the classifier** using **precision** and **recall** from last week, and see where a single straight line runs out of road: the opposite-corners **XOR** pattern that no line can split.

Every step is arithmetic you have already done by hand in these lecture notes: multiplying, adding, and reading off a sign. The only thing you set yourself is the weights; watching the boundary move as you change them is the whole point.